

DECODELABS

PROJECT SUBMISSION REPORT

Rule-Based AI Chatbot

A Python Implementation Using Conditional Logic, Iteration, Dictionaries, and Regular Expressions

Project Title	Rule-Based AI Chatbot
Language / Tools	Python 3, re, json, datetime, tkinter
Category	Artificial Intelligence — Rule-Based Systems
Submitted To	DecodeLabs
Submission Date	18 June 2026

Table of Contents

Table of Contents.....	2
1. Introduction.....	3
1.1 Objective.....	3
1.2 Scope.....	3
2. System Requirements.....	4
2.1 Software Requirements.....	4
2.2 Hardware Requirements.....	4
3. System Design.....	5
3.1 Design Rationale.....	5
4. Flowcharts.....	6
4.1 Overall System Flow.....	6
4.2 Rule-Matching Engine Logic.....	7
5. Source Code.....	8
5.1 Imports and Rule Dictionary.....	8
5.2 Default Fallback Responses.....	8
5.3 The Rule-Matching Engine — <code>get_response()</code>	9
5.4 Persistent JSON Logging.....	9
5.5 Console Chat Loop.....	10
5.6 Program Entry Point.....	11
6. Testing and Sample Output.....	12
6.1 Test Case Summary.....	12
6.2 Persisted Conversation Log.....	13
7. Graphical User Interface (Tkinter).....	14
7.1 GUI Features.....	14
8. Conclusion.....	15
8.1 Future Scope.....	15
9. References.....	16

1. Introduction

Conversational software agents, commonly called chatbots, are programs designed to simulate human-like dialogue. This project implements a rule-based chatbot in Python — a system that selects responses by matching user input against a predefined set of patterns rather than relying on machine learning or external AI services.

Rule-based chatbots remain widely used in production systems such as customer support widgets, FAQ assistants, and IVR (Interactive Voice Response) systems because they are predictable, transparent, lightweight, and easy to extend without requiring training data or a model-hosting infrastructure.

1.1 Objective

The objective of this project is to design and implement a chatbot that:

- Responds to user messages using keyword and pattern-based rules stored in a Python dictionary.
- Uses regular expressions for flexible, whole-word matching instead of brittle substring checks.
- Scores multiple rule matches and selects the best-fitting response rather than the first match found.
- Persists every conversation exchange to a JSON log file with timestamps.
- Offers both a console (command-line) interface and a graphical (Tkinter) interface.

1.2 Scope

The chatbot covers a small set of conversational intents — greetings, farewells, small talk, jokes, and live time/date queries — sufficient to demonstrate the core rule-based architecture. The same architecture can be extended to larger rule sets for domain-specific assistants (e.g., a college helpdesk bot or a product FAQ bot) without any structural changes.

2. System Requirements

2.1 Software Requirements

Requirement	Details
Language	Python 3.6 or higher
Built-in Modules	re, json, os, datetime, random
GUI Toolkit	tkinter (bundled with standard Python installation)
Operating System	Windows, macOS, or Linux
IDE / Editor	Any text editor, VS Code, PyCharm, or terminal-based execution

2.2 Hardware Requirements

Requirement	Details
Processor	Any modern CPU (1 GHz or above)
RAM	512 MB minimum (negligible footprint)
Storage	Less than 1 MB for source code and logs

3. System Design

The chatbot follows a simple three-layer design: an Interface Layer (console or GUI) captures user input; a Rule-Matching Engine scores that input against a dictionary of regular-expression patterns; and a Persistence Layer logs every exchange to disk. This separation keeps the matching logic independent of how the user interacts with it — the same `get_response()` function powers both interfaces.

Figure 3: Module / Component Architecture

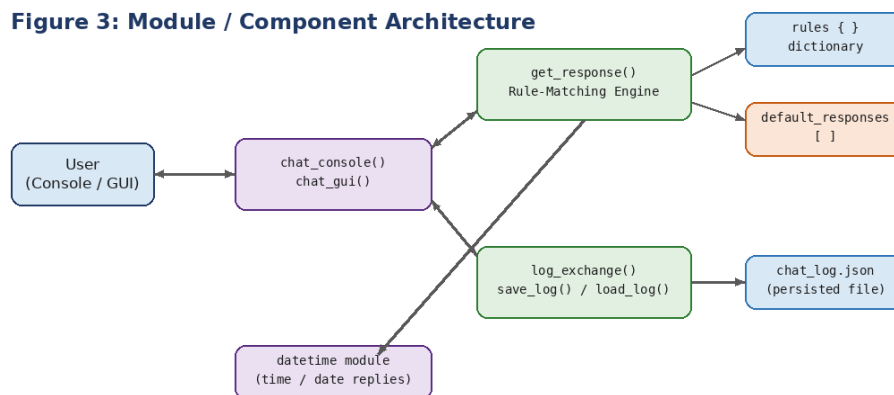


Figure 3: Module and component architecture of the chatbot.

3.1 Design Rationale

- Dictionary-based rules: keeps all conversational knowledge in one data structure, making it trivial to add, remove, or edit intents without touching control-flow code.
- Regular expressions over plain substring checks: `\b...\b` word boundaries prevent false positives (e.g., the word “this” no longer accidentally matches a rule for “hi”).
- Scoring instead of first-match: if a user message matches multiple rules, the rule with the most pattern hits is chosen, giving more relevant replies on compound queries.
- JSON logging: human-readable, easily parsed by other tools, and naturally append-friendly across sessions.

4. Flowcharts

Two flowcharts describe the system: the first shows the overall conversation loop from program start to exit; the second zooms into the rule-matching engine that selects which response to return.

4.1 Overall System Flow

Figure 1: Chatbot System Flowchart

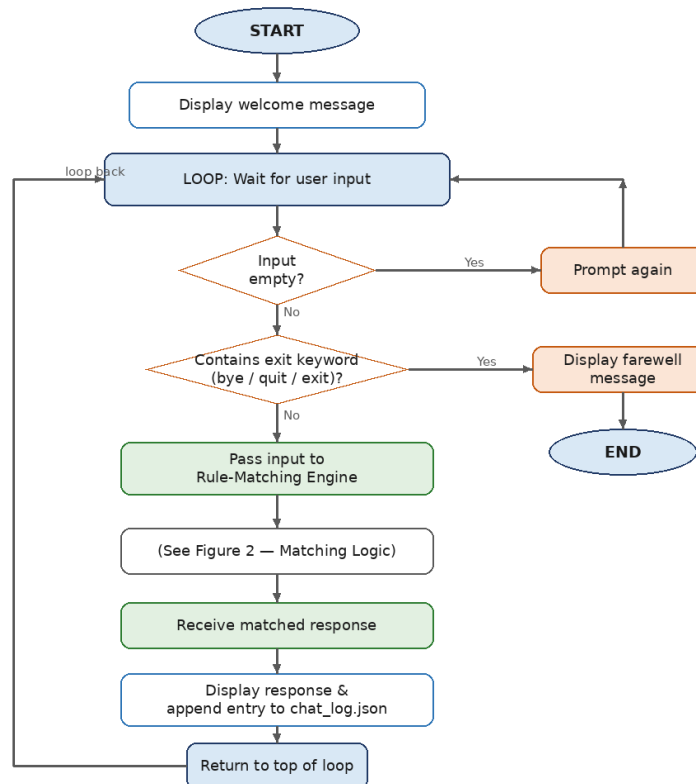


Figure 1: End-to-end conversation flow, from program start to exit.

4.2 Rule-Matching Engine Logic

This flowchart expands the “Pass input to Rule-Matching Engine” step from Figure 1. Every rule pattern is tested against the input; the pattern producing the highest number of regex matches wins, and a random reply from its associated list is returned. If no pattern matches at all, a default fallback response is used instead.

Figure 2: Rule-Matching Engine (Scoring Logic)

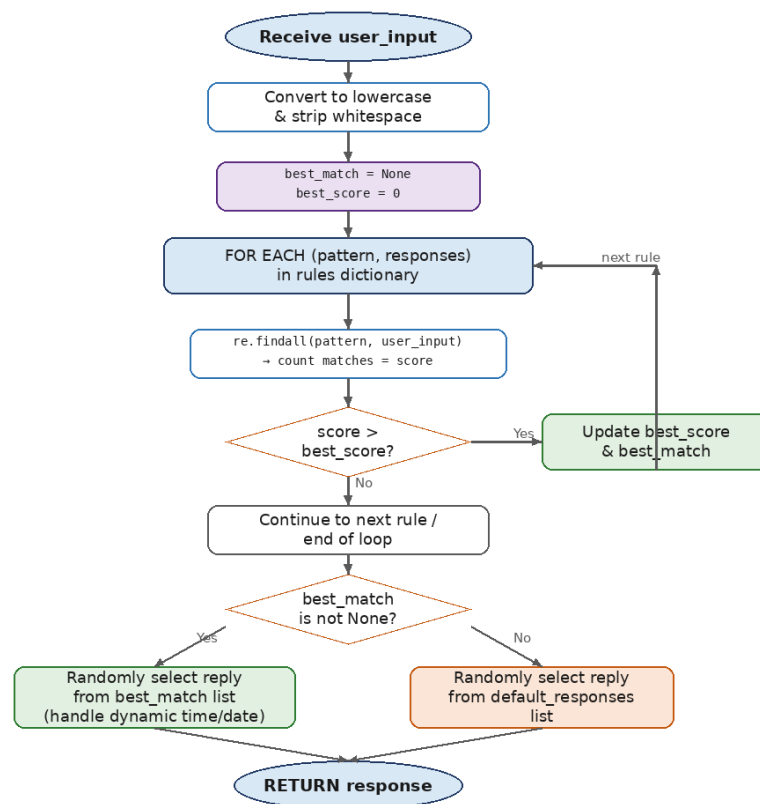


Figure 2: Detailed logic inside `get_response()`, the rule-scoring function.

5. Source Code

The complete implementation is presented below, organized by functional block. Line numbers correspond to the full chatbot.py file.

5.1 Imports and Rule Dictionary

The rules dictionary maps a regular-expression pattern (as the key) to a list of possible replies (as the value). Using `\b` word-boundary markers ensures whole-word matching — for example, `\b(hello|hi|hey)\b` will not accidentally trigger on a word like “thistle”.

```
import random
import re
import json
from datetime import datetime
import os

# -----
# 1. RULES: keyword patterns (regex) -> responses
# -----
rules = {
    r"\b(hello|hi|hey)\b": ["Hi there!", "Hello!", "Hey! How can I help you?"],
    r"\bhow are you\b": ["I'm just a program, but I'm doing great!", "I'm functioning as expected!"],
    r"\b(your name|who are you)\b": ["I'm a rule-based chatbot built in Python.", "You can call me ChatBot."],
    r"\b(your age|how old)\b": ["I don't have an age, I'm just code!", "Age doesn't apply to me."],
    r"\bweather\b": ["I can't check live weather, but I hope it's nice where you are!"],
    r"\b(help|what can you do)\b": ["I can chat about basic topics. Try asking about my name, age, or say hello!"],
    r"\b(joke|funny)\b": [
        "Why don't programmers like nature? It has too many bugs.",
        "Why do Python programmers wear glasses? Because they can't C."
    ],
    r"\b(thanks|thank you)\b": ["You're welcome!", "No problem!", "Anytime!"],
    r"\btime\b": ["__TIME__"],
    r"\b(date|today)\b": ["__DATE__"],
    r"\b(bye|goodbye|exit|quit)\b": ["Goodbye!", "See you later!", "Bye! Take care."]
}
```

5.2 Default Fallback Responses

When no rule pattern matches the user's input at all, a random reply is chosen from this fallback list so the bot never returns an empty or broken response.

```
default_responses = [
    "I'm not sure I understand. Can you rephrase that?",
    "Could you elaborate on that?",
    "I don't have an answer for that yet.",
    "Hmm, I'm not trained to answer that."
]

LOG_FILE = "chat_log.json"
```

5.3 The Rule-Matching Engine — `get_response()`

This function implements the scoring algorithm shown in Figure 2. It iterates over every (pattern, responses) pair, counts how many times each pattern matches the cleaned input using `re.findall()`, and keeps track of the highest-scoring rule seen so far.

```
def get_response(user_input):
    user_input = user_input.lower().strip()
    best_match = None
    best_score = 0

    for pattern, possible_responses in rules.items():
        matches = re.findall(pattern, user_input)
        score = len(matches)
        if score > best_score:
            best_score = score
            best_match = possible_responses

    if best_match:
        reply = random.choice(best_match)
        if reply == "__TIME__":
            reply = f"It's {datetime.now().strftime('%H:%M:%S')} right now."
        elif reply == "__DATE__":
            reply = f"Today is {datetime.now().strftime('%A, %B %d, %Y')}."
        return reply

    return random.choice(default_responses)
```

5.4 Persistent JSON Logging

Three small functions handle reading the existing log (if any), writing the full log back to disk, and appending a single timestamped exchange. Using `json.dump` with `indent=2` keeps the log file human-readable.

```
def load_log():
    if os.path.exists(LOG_FILE):
```

```
        with open(LOG_FILE, "r") as f:
            try:
                return json.load(f)
            except json.JSONDecodeError:
                return []
    return []

def save_log(log):
    with open(LOG_FILE, "w") as f:
        json.dump(log, f, indent=2)

def log_exchange(log, user_msg, bot_msg):
    log.append({
        "timestamp": datetime.now().isoformat(timespec="seconds"),
        "user": user_msg,
        "bot": bot_msg
    })
    save_log(log)
```

5.5 Console Chat Loop

The `chat_console()` function drives the main interaction loop: it prints a welcome banner, then repeatedly reads input, checks for an exit keyword, and otherwise delegates to `get_response()` before logging and printing the reply.

```
def chat_console():
    print("ChatBot: Hello! Type 'quit' or 'exit' to end the chat.\n")
    log = load_log()

    while True:
        user_input = input("You: ")

        if user_input.strip() == "":
            print("ChatBot: Please type something!")
            continue

        if re.search(r"\b(bye|goodbye|exit|quit)\b", user_input.lower()):
            response = random.choice(["Goodbye!", "See you later!", "Bye! Take care."])
            print("ChatBot:", response)
            log_exchange(log, user_input, response)
            break

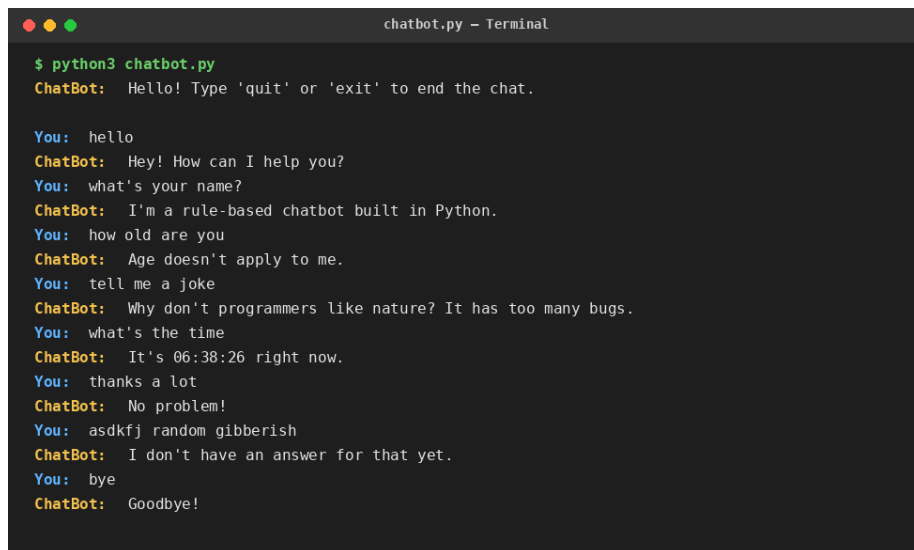
        response = get_response(user_input)
        print("ChatBot:", response)
```

5.6 Program Entry Point

```
if __name__ == "__main__":  
    chat_console()
```

6. Testing and Sample Output

The chatbot was executed in console mode and exercised against a range of inputs covering greetings, factual questions, jokes, dynamic time queries, gratitude expressions, and unrecognized input. The screenshot below is a direct capture of the actual program output, not a simulated transcript.

A screenshot of a terminal window titled "chatbot.py - Terminal". The terminal shows a series of interactions between a user and a chatbot. The user inputs are in blue, and the chatbot responses are in orange. The chatbot starts with a greeting and instructions on how to end the chat. It then responds to various user inputs including greetings, questions about its name and age, requests for jokes, time queries, gratitude, random gibberish, and a goodbye message.

```
chatbot.py - Terminal
$ python3 chatbot.py
ChatBot: Hello! Type 'quit' or 'exit' to end the chat.

You: hello
ChatBot: Hey! How can I help you?
You: what's your name?
ChatBot: I'm a rule-based chatbot built in Python.
You: how old are you
ChatBot: Age doesn't apply to me.
You: tell me a joke
ChatBot: Why don't programmers like nature? It has too many bugs.
You: what's the time
ChatBot: It's 06:38:26 right now.
You: thanks a lot
ChatBot: No problem!
You: asdkfj random gibberish
ChatBot: I don't have an answer for that yet.
You: bye
ChatBot: Goodbye!
```

Figure 4: Console session output — captured directly from a live run of chatbot.py.

6.1 Test Case Summary

User Input	Matching Behavior	Bot Response
hello	Keyword match (greeting)	Hey! How can I help you?
what's your name?	Keyword match (identity)	I'm a rule-based chatbot built in Python.
how old are you	Keyword match (age)	Age doesn't apply to me.
tell me a joke	Keyword match (joke)	Why don't programmers like nature? It has too many bugs.
what's the time	Dynamic response (live time)	It's 06:38:26 right now.
thanks a lot	Keyword match (gratitude)	No problem!
asdkfj random gibberish	No rule matched → fallback	I don't have an answer for that yet.
bye	Exit keyword detected	Goodbye! (loop terminates)

6.2 Persisted Conversation Log

Every exchange shown above was simultaneously written to `chat_log.json` with an ISO-format timestamp, confirming that the persistence layer operates correctly alongside the matching engine.

```
[
  {
    "timestamp": "2026-06-18T06:38:26",
    "user": "hello",
    "bot": "Hey! How can I help you?"
  },
  {
    "timestamp": "2026-06-18T06:38:26",
    "user": "what's your name?",
    "bot": "I'm a rule-based chatbot built in Python."
  },
  {
    "timestamp": "2026-06-18T06:38:26",
    "user": "how old are you",
    "bot": "Age doesn't apply to me."
  },
  ...
]
```

Figure 5: Excerpt of the auto-generated `chat_log.json` file from the test run above.

7. Graphical User Interface (Tkinter)

In addition to the console interface, the project includes an optional Tkinter-based GUI accessible via the `chat_gui()` function. It reuses the exact same `get_response()` engine and logging functions as the console mode — only the input/output layer changes — which demonstrates the benefit of separating the rule-matching logic from the interface.

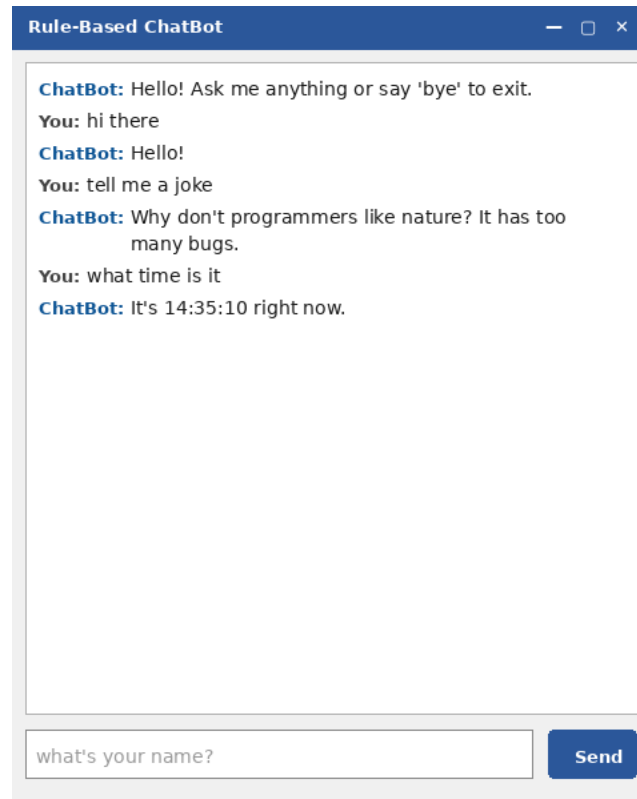


Figure 6: Tkinter chat window showing a live conversation with scrollable history.

7.1 GUI Features

- Scrollable chat history area showing the full conversation.
- Single-line text entry with Enter-to-send support, in addition to a Send button.
- Automatic window close shortly after a farewell keyword is detected.
- Identical logging behavior to console mode — every GUI exchange is also saved to `chat_log.json`.

8. Conclusion

This project successfully demonstrates a rule-based AI chatbot built entirely with Python's standard library. By combining a dictionary of regular-expression rules, a scoring algorithm, and simple persistence, the chatbot can hold short, coherent conversations without any machine-learning dependency. The dual console/GUI interface and JSON logging make the system practical for both quick terminal testing and a more user-friendly desktop experience.

The project reinforced core Python concepts requested for this submission — conditional logic (if-else), iteration (for and while loops), and dictionaries — while also introducing complementary techniques (regular expressions, randomized responses, and file-based persistence) that are common in real-world rule-based systems.

8.1 Future Scope

- Replace the in-memory dictionary with rules loaded from an external JSON/YAML configuration file, so non-programmers can add new intents.
- Introduce simple context tracking (e.g., remembering the user's name across a session).
- Add a lightweight intent-confidence threshold to avoid weak matches being treated as confident answers.
- Integrate optional NLP libraries (e.g., spaCy) for lemmatization, enabling the bot to match “running” and “ran” to the same rule.
- Deploy the chatbot as a web service using Flask, exposing `get_response()` through a REST API.

9. References

1. Python Software Foundation. Python 3 Documentation — re (Regular expression operations). [docs.python.org](https://docs.python.org/3/library/re.html).
2. Python Software Foundation. Python 3 Documentation — json (JSON encoder and decoder). [docs.python.org](https://docs.python.org/3/library/json.html).
3. Python Software Foundation. Python 3 Documentation — tkinter (Tk GUI toolkit). [docs.python.org](https://docs.python.org/3/library/tkinter.html).
4. Python Software Foundation. Python 3 Documentation — datetime (Basic date and time types). [docs.python.org](https://docs.python.org/3/library/datetime.html).
5. Russell, S., & Norvig, P. Artificial Intelligence: A Modern Approach — chapter on rule-based and reflex agents.